

Enhancing ABAC Performance and Scalability on Fedora

Final Project report submitted to
Indian Institute of Technology Kharagpur
in partial fulfilment for the award of the degree of
Bachelor of Technology
in
Computer Science and Engineering

by
Ojas Dubey
(22CS30039)

Under the supervision of
Professor Shamik Sural



Department of Computer Science and Engineering

Indian Institute of Technology Kharagpur

Spring Semester 2025–26

April 2026

DECLARATION

I certify that

- (a) The work contained in this report has been done by me under the guidance of my supervisor.
- (b) The work has not been submitted to any other Institute for any degree or diploma.
- (c) I have conformed to the norms and guidelines given in the Ethical Code of Conduct of the Institute.
- (d) Whenever I have used materials (data, theoretical analysis, figures, and text) from other sources, I have given due credit to them by citing them in the text of the thesis and giving their details in the references. Further, I have taken permission from the copyright owners of the sources, whenever necessary.

Date: April 2026
Place: Kharagpur

(Ojas Dubey)
(22CS30039)

DEPARTMENT OF COMPUTER SCIENCE AND
ENGINEERING
INDIAN INSTITUTE OF TECHNOLOGY KHARAGPUR
KHARAGPUR - 721302, INDIA



CERTIFICATE

This is to certify that the project report entitled “Enhancing ABAC Performance and Scalability on Fedora” submitted by Ojas Dubey (Roll No. 22CS30039) to Indian Institute of Technology Kharagpur towards partial fulfilment of requirements for the award of degree of Bachelor of Technology in Computer Science and Engineering is a record of bona fide work carried out by him under my supervision and guidance during Spring Semester 2025–26.

Date: April 2026

Place: Kharagpur

Professor Shamik Sural
Department of Computer Science and
Engineering
Indian Institute of Technology Kharagpur
Kharagpur - 721302, India

Abstract

Name of the student: **Ojas Dubey**

Roll No: **22CS30039**

Degree for which submitted: **Bachelor of Technology**

Department: **Department of Computer Science and Engineering**

Thesis title: **Enhancing ABAC Performance and Scalability on Fedora**

Thesis supervisor: **Professor Shamik Sural**

Month and year of thesis submission: **April 2026**

This report presents the design, porting, implementation, and evaluation of an Attribute-Based Access Control (ABAC) Linux Security Module (LSM) on a modern Fedora-oriented Linux ‘6.19.8’ workflow. The work begins from the original ABAC-LSM research lineage, which established the feasibility of enforcing ABAC for file-system objects through Linux Security Modules. Building on those foundations, the repository modernizes the kernel integration path, packages the maintained kernel work as a reusable patch, and extends the original design with explicit negative rules, a compiled policy-tree resolver, hash-map indexed user and object lookup, runtime feature flags for tree and hash optimizations, kernel-side performance instrumentation, configurable include/exclude path scope, directory-level object attribute inheritance, and a practical benchmark harness for the active maintained implementation.

The resulting system preserves the core ABAC architecture of kernel-side enforcement through the `file_permission` hook and userspace-to-kernel state loading through `securityfs`, while making the system significantly easier to operate, test, and evaluate on a current kernel. The userspace layer includes a CLI tool, installation workflow, background services, scope management support, and

environment-attribute refresh logic. The benchmarking workflow generates synthetic datasets, loads them into the kernel, executes real non-root access attempts, and is structured to compare a four-way optimization grid formed by enabling or disabling the policy-tree resolver and hash-map indexed lookup independently.

Functional validation showed that the ABAC LSM boots correctly, appears in the active LSM list, exposes the expected `/sys/kernel/security/abac` interface, and enforces policy as intended. End-to-end tests confirmed correct handling of allow/deny behavior, exclusion rules, and directory-level inheritance. Performance evaluation demonstrated that the compiled tree-based resolver reduces mean user-observed access latency from 22,896.65 ns to 16,962.23 ns, an improvement of approximately 25.9%, while kernel-side decision time decreases from 15,970,257 ns to 13,483,783 ns, an improvement of approximately 15.6%. These results show that the modernized port preserves the architectural intent of earlier ABAC-LSM work while extending it into a more deployable and experimentally useful system.

Acknowledgements

It is a matter of great pleasure to submit this project report on “Enhancing ABAC Performance and Scalability on Fedora,” prepared from the maintained ABAC Fedora port repository as part of the academic work of the Department of Computer Science and Engineering, IIT Kharagpur. The report consolidates the implementation, porting, testing, and evaluation effort carried out across the kernel patch, userspace utilities, and performance study. I take this opportunity to acknowledge the guidance, supporting materials, and prior ABAC-LSM research foundations that made it possible to bring the work together in a single report.

I acknowledge the use of GPT-5.4 for minor refinements in this report.

Contents

Declaration	i
Certificate	ii
Abstract	iii
Acknowledgements	v
Contents	vi
Abbreviations	viii
List of Tables	ix
List of Figures	xi
1 Introduction	1
1.1 Background	1
1.2 Problem Statement	2
1.3 Motivation	2
2 Literature Review	4
2.1 Enabling Attribute-Based Access Control in Linux Kernel	4
2.1.1 Background and Introduction	4
2.1.2 Methodology	5
2.1.3 Findings and Results	5
2.1.4 Critical Analysis	5
2.1.5 Relevance to the Present Research	6
2.2 Efficiently Supporting Attribute-Based Access Control in Linux	6
2.2.1 Background and Introduction	6
2.2.2 Methodology	7
2.2.3 Findings and Results	7
2.2.4 Critical Analysis	8
2.2.5 Relevance to the Present Research	8

2.3	How the Current Repository Builds on the Literature	8
3	Methodology	10
3.1	System Overview	10
3.2	Kernel–Userspace Architecture	11
3.3	Resolver Design	12
3.4	Coverage and Object-Model Design	15
3.5	Benchmark Methodology	17
4	Implementation Details	20
4.1	Kernel Patch and Porting Changes	20
4.1.1	LSM Registration Port	21
4.1.2	Securityfs Initialization Port	21
4.1.3	Correctness Fixes Included in the Patch	22
4.1.4	Runtime Interface Added Through the Patch	22
4.2	Userspace Utilities and Services	23
4.3	Negative Rule Support	23
4.4	Compiled Policy Tree and Runtime Optimization Flags	24
4.5	Path Scope Support	29
4.6	Directory-Level Object Attribute Inheritance	31
4.7	Kernel-Side Instrumentation	32
4.8	Benchmark Harness Implementation	32
4.9	Files Central to the New Features	33
4.10	Design Decisions	34
4.11	Challenges Encountered	35
4.12	Hash-Map Indexed User and Object Lookup	37
5	Results	39
5.1	Functional Validation	39
5.2	Authorization Behavior	39
5.3	Performance Evaluation	40
5.4	Discussion of Results	41
6	Conclusion	43
6.1	Key Outcomes	43
6.2	Next Steps	44
	Bibliography	45

Abbreviations

ABAC	A tttribute- B ased A ccess C ontrol
ACL	A ccess C ontrol L ist
DAC	D iscretionary A ccess C ontrol
LSM	L inux S ecurity M odule
PDP	P olicy D ecision P oint
PEP	P olicy E nforcement P oint
RBAC	R ole- B ased A ccess C ontrol
VFS	V irtual F ile S ystem

List of Tables

4.1	Feature-wise contribution map for the maintained ABAC Fedora port	34
5.1	Optimization-grid performance results from the maintained Fedora port	40
5.2	Interpretation of the maintained feature set by technical impact . . .	41

List of Algorithms

1	Access Decision Workflow in the Maintained ABAC LSM	27
---	---	----

List of Figures

3.1	High-level organization of the maintained ABAC Fedora port.	11
3.2	Kernel–userspace data flow from persistent configuration to runtime enforcement.	12
3.3	Resolver-level workflow showing that hash-indexed state lookup and tree-based policy evaluation are controlled independently.	14
3.4	Coverage decision flow showing how the maintained design expands from one fixed protected directory to include/exclude-based runtime coverage.	15
3.5	Directory-level object modeling in the maintained design. Parent directories contribute inherited attributes, while excluded branches leave the ABAC path entirely.	16
3.6	Benchmark orchestration showing the experimental pipeline around the 2x2 optimization sweep.	18
3.7	Four-configuration optimization grid used for planned performance evaluation.	19

Chapter 1

Introduction

1.1 Background

Linux provides strong baseline protection through discretionary access control, groups, and access control lists, and it further supports mandatory access control extensions through the Linux Security Module framework. However, the traditional Linux access-control model remains less expressive than Attribute-Based Access Control when policies must depend simultaneously on user properties, object properties, and environmental conditions. ABAC is attractive because it supports fine-grained, centrally managed policy without the role explosion associated with RBAC or the repetitive permission management typical of DAC.

Previous ABAC research showed that Linux can support ABAC for file-system objects by implementing the enforcement and decision path inside the kernel and using `securityfs` to load policies and attributes from userspace. The current repository inherits that architecture, but targets a practical Fedora-oriented path centered on a modern Linux ‘6.19.8’ kernel. Instead of preserving an entire patched kernel tree, it packages the maintained kernel-side work as a patch and keeps

the supporting userspace utilities and benchmarking tools in a compact repository structure.

1.2 Problem Statement

The core problem addressed in this report is how to preserve and extend ABAC-on-Linux functionality in a way that is maintainable on a modern kernel while also remaining experimentally meaningful. The original code path provided a useful proof of concept, but it had several practical limitations: policy resolution was essentially linear, scope was effectively tied to a hard-coded directory model, object attributes had to be managed at the file level, and the benchmarking workflow was tied more closely to older research variants than to the active maintained implementation.

This work therefore asks the following questions. First, how can the original ABAC-LSM design be ported to a modern Linux kernel without losing the essential architecture? Second, how can the maintained implementation be extended so that it supports richer policies and more usable administrative behavior? Third, how can the active code path be benchmarked fairly so that different resolver strategies can be compared under the same kernel and workload?

1.3 Motivation

The motivation for this report lies in bridging the gap between research prototype and maintainable system. A modern ABAC-LSM implementation is valuable only if it can be built on a current kernel, operated through a usable userspace toolchain, and evaluated through a benchmark path that reflects the code actually being maintained. That required not only porting old interfaces forward, but also rethinking how scope, inheritance, instrumentation, and benchmark orchestration should work in the present repository.

There is also a performance motivation. The literature makes clear that while ABAC is feasible in Linux, resolver design strongly affects overhead. This repository therefore treats resolver structure as a first-class implementation concern, preserving a direct comparison between **linear** and **tree** resolution while keeping the rest of the stack fixed. In that sense, the report is both a systems-integration report and a continuation of the original performance-oriented ABAC work.

Chapter 2

Literature Review

2.1 Enabling Attribute-Based Access Control in Linux Kernel

2.1.1 Background and Introduction

The Asia CCS 2022 paper *Enabling Attribute-Based Access Control in Linux Kernel* provides the conceptual starting point for the repository [1]. Its central contribution is the claim that ABAC can be incorporated into Linux file-system enforcement without modifying the core VFS path directly and without routing every access decision through a slower userspace decision service. Instead, the paper positions the Linux Security Module framework as the natural location for the policy enforcement and decision path.

This is important because it establishes the broad architectural assumptions retained in the current repository. The kernel is responsible for enforcement. Userspace is responsible for persistence, management, and state loading. `securityfs` bridges the two. The `file_permission` hook is used for access mediation. Objects are identified by path, and policies are written over user, object, and environmental attributes.

2.1.2 Methodology

The 2022 paper proposes a four-part system: an ABAC LSM, a `securityfs` interface, userspace background services, and a CLI administration tool. It also discusses alternatives that were not chosen, such as using FUSE or relying on userspace communication paths like netlink for policy resolution. The paper argues that policy decisions must stay inside the kernel if latency is to remain acceptable and if enforcement is to remain well aligned with the Linux access-control model.

This architectural structure maps directly to the present repository. The maintained kernel-side implementation is represented by a patch that wires `security/abac` into a clean Linux ‘6.19.8’ tree. The userspace layer exists in `abac_utils/`. Live ABAC state is exposed through `/sys/kernel/security/abac/`. Persistent userspace state is stored under `/etc/abac/`.

2.1.3 Findings and Results

The paper reported that the original linked-list-based ABAC design was practical even on Linux ‘5.10.62’. The main reported pattern was that access time rose roughly linearly with policy size, but remained acceptable for the problem sizes evaluated. The key result was therefore not ultimate efficiency, but feasibility: ABAC could be integrated into Linux in a way that worked end to end for file-system policy.

2.1.4 Critical Analysis

The main strength of the 2022 paper is that it establishes the architectural legitimacy of ABAC inside Linux. It identifies the right extension points, argues convincingly for in-kernel policy resolution, and demonstrates that userspace services and `securityfs` together form a workable control plane. Its main limitation is that it leaves many practical concerns underdeveloped for long-term maintenance,

including modern kernel API compatibility, more flexible coverage scoping, richer policy semantics, and benchmarking aligned with the actively maintained path.

2.1.5 Relevance to the Present Research

The present repository directly inherits the 2022 architectural decisions. The Fedora port keeps the same LSM-centered enforcement model, the same general kernel/userspace split, and the same path-based object view, but extends the original design with a patch-based modern integration path, configurable include/exclude scope, directory-level object inheritance, and cleaner deployment and evaluation support.

2.2 Efficiently Supporting Attribute-Based Access Control in Linux

2.2.1 Background and Introduction

The IEEE TDSC 2024 paper *Efficiently Supporting Attribute-Based Access Control in Linux* picks up from the earlier architecture and asks how resolver performance can be improved [2]. Its emphasis is not primarily on feasibility but on efficiency: given that ABAC can be supported in Linux, what data structures and evaluation strategies make that support practical as policies grow?

The paper therefore shifts attention from system placement to resolver organization. It highlights that repeated policy scanning is costly and that policy representation matters. This is the part of the literature that most directly informs the current repository’s runtime resolver modes and benchmarking story.

2.2.2 Methodology

The paper compares different resolution strategies, especially rule-list based evaluation and Access Control Tree based evaluation. The policy tree idea is used to reduce repeated comparisons and organize policy conditions in a structure that is cheaper to traverse than repeatedly scanning all matching rules.

The maintained Fedora port builds on that insight but does not reproduce the paper’s research variants exactly. Instead, it uses a simpler maintained compiled tree organized around object buckets, operation buckets, user/environment match nodes, and effect leaves. This keeps the active kernel path smaller and easier to maintain while preserving the same underlying objective of reducing repeated rule traversal. In addition, the maintained resolver pairs this compiled tree with hash-map indexed user and object lookup, so the optimization applies not only to policy evaluation itself but also to the state-fetch stage that precedes evaluation.

Although the current repository does not reproduce every historical research variant discussed in the research lineage, it clearly inherits the methodological direction of the paper. The active maintained path exposes `linear` and `tree` modes inside one kernel implementation, allowing the repository to compare resolution strategy while holding the rest of the system fixed.

2.2.3 Findings and Results

The 2024 paper reported much stronger efficiency evidence than the 2022 work. It showed that tree-based evaluation was faster than rule-list based evaluation and that the more structured methods substantially outperformed the earlier single-policy baseline. The paper also found that policy size and user attribute count are important performance drivers, while some other workload dimensions are less significant when data access is organized effectively.

2.2.4 Critical Analysis

The main strength of the 2024 paper is that it reframes ABAC-LSM implementation as a data-structure problem rather than just a hooking problem. That perspective is crucial for a maintained system. Its limitation, from the present repository's perspective, is that research-oriented prototypes often contain multiple variants and experimental formats that are not all ideal for a compact modern maintenance path. The current repository therefore adopts the core insight while simplifying the active implementation surface.

2.2.5 Relevance to the Present Research

The maintained Fedora port directly extends the performance logic of the 2024 paper. It keeps explicit resolver-mode comparison, kernel-side instrumentation, and synthetic workload generation, but repackages them into a more focused maintained implementation. The result is a smaller active system that still preserves the key experimental question: how much improvement does structured tree-based evaluation provide over sequential rule scanning, and how much additional benefit comes from hash-map indexed access to user and object state?

2.3 How the Current Repository Builds on the Literature

Taken together, the two papers define the intellectual lineage of the repository. The first contributes the architecture. The second contributes the efficiency direction. The present repository builds on both through several concrete additions:

- a modern Linux ‘6.19.8’ patch-based integration path

- explicit negative-rule support
- a compiled policy-tree resolver maintained in the active code path
- hash-map indexed user and object lookup integrated with the resolver path
- runtime mode switching between `linear` and `tree`
- kernel-exposed performance instrumentation through `action`, `perf`, `stats`, and `mode`
- configurable include/exclude path scope
- directory-level object attribute inheritance
- a benchmark harness aligned with the maintained kernel interface

In that sense, the repository is best understood not as a reimplementation of the papers, but as a continuation of them: it modernizes their architecture, retains their performance concerns, and packages them into a form suitable for both practical use and further experimentation.

Chapter 3

Methodology

3.1 System Overview

The ABAC Fedora port consists of three main operational layers: the kernel-side LSM integration, the userspace control plane, and the benchmarking/evaluation tooling. The kernel side is responsible for interception and enforcement. The userspace layer is responsible for persistent configuration, policy and attribute loading, object-attribute management, environmental updates, and installation/service wiring. The performance tooling is responsible for generating workloads, switching resolver modes, invoking real file accesses under a non-root user, and collecting timing statistics.

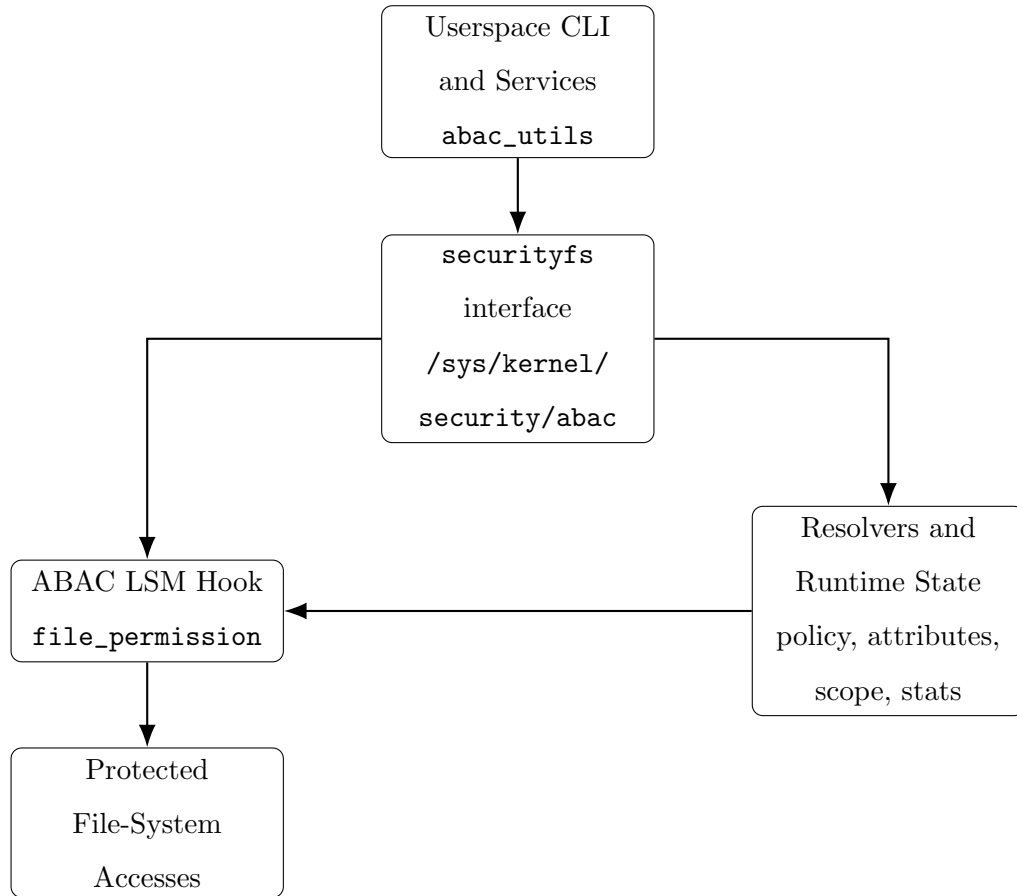


FIGURE 3.1: High-level organization of the maintained ABAC Fedora port.

3.2 Kernel–Userspace Architecture

The actively deployable kernel-side integration is preserved as a patch in `kernel_patch/abac-linux-6.19.8.patch`. Rather than keep a full modified kernel tree in version control, the repository stores the ABAC work as a reusable patch that can be applied to a pristine Linux ‘6.19.8’ source tree. This keeps the repository compact and makes the kernel contribution explicit as a bounded set of changes.

At a design level, the patch has four roles:

- make ABAC visible to the kernel build system

- insert the maintained `security/abac` source tree into the kernel
- reconcile older ABAC code with modern LSM and initcall interfaces
- expose a richer runtime control plane through `securityfs`

The userspace layer complements this kernel path. It persists policy and attribute state under `/etc/abac/`, serializes that state into `/sys/kernel/security/abac/`, and runs background services for object-attribute management, directory watching, and environment updates. In other words, the kernel provides enforcement and runtime state, while userspace provides persistence, administration, and refresh logic.

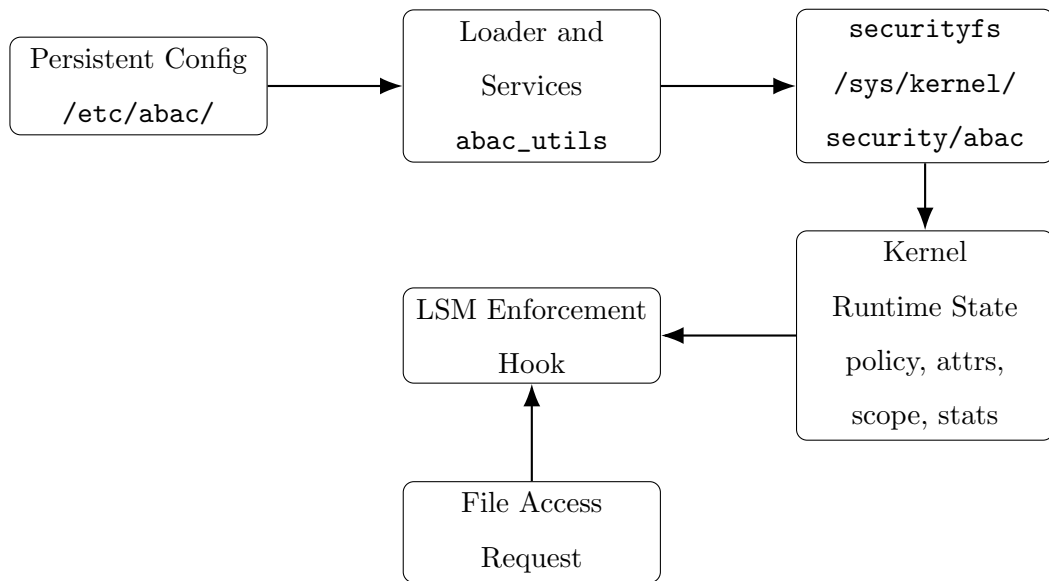


FIGURE 3.2: Kernel–userspace data flow from persistent configuration to runtime enforcement.

3.3 Resolver Design

The core methodological decision in the maintained implementation is to separate policy semantics from resolver strategy. Policy is still written in a flat textual form,

but the maintained kernel is structured around two orthogonal optimization controls over that same policy:

- a policy-tree control that selects either sequential rule-list evaluation or compiled tree evaluation
- a hash-index control that enables or disables hash-map indexed user and object lookup

This yields a four-configuration evaluation grid: neither optimization enabled, only the tree enabled, only the hash-map index enabled, and both enabled together. The key design goal is fair comparison. The same policy language, kernel build, object attributes, user attributes, and environment attributes are used in every configuration. Only the optimization flags change. This makes the resolver comparison meaningful and also reduces maintenance complexity.

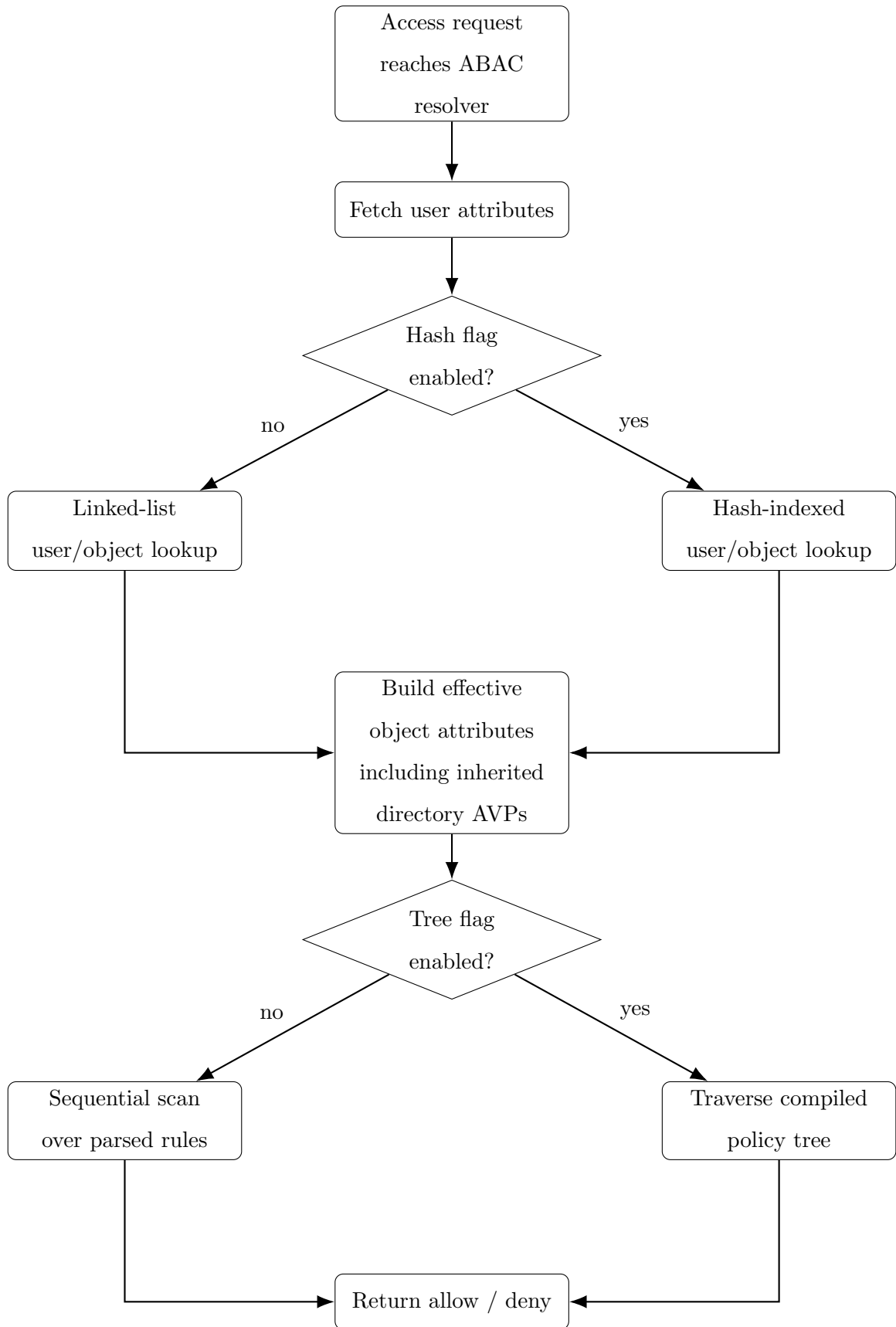


FIGURE 3.3: Resolver-level workflow showing that hash-indexed state lookup and tree-based policy evaluation are controlled independently.

3.4 Coverage and Object-Model Design

The maintained implementation also makes two important methodological changes to the administrative model. First, ABAC coverage is no longer tied to a single built-in protected directory. Instead, the runtime system supports include/exclude prefix lists. Second, object attributes can be inherited from directories down to descendants, with more specific paths overriding less specific values for the same attribute name.

These changes are design-level improvements rather than only code-level conveniences. They make the system much closer to a deployable file-system policy mechanism instead of a narrowly scoped proof of concept.

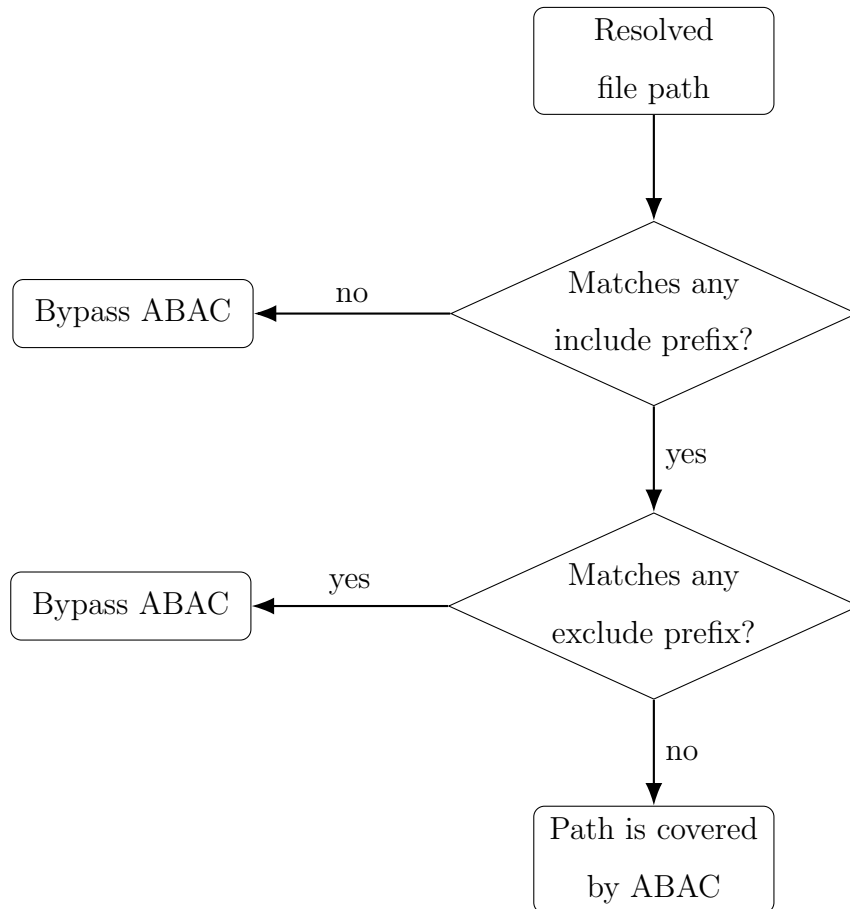


FIGURE 3.4: Coverage decision flow showing how the maintained design expands from one fixed protected directory to include/exclude-based runtime coverage.

This coverage step is the point where the old single-directory model becomes a reusable runtime mechanism. Every candidate path is first checked against include/exclude scope, and only covered paths continue into the resolver.

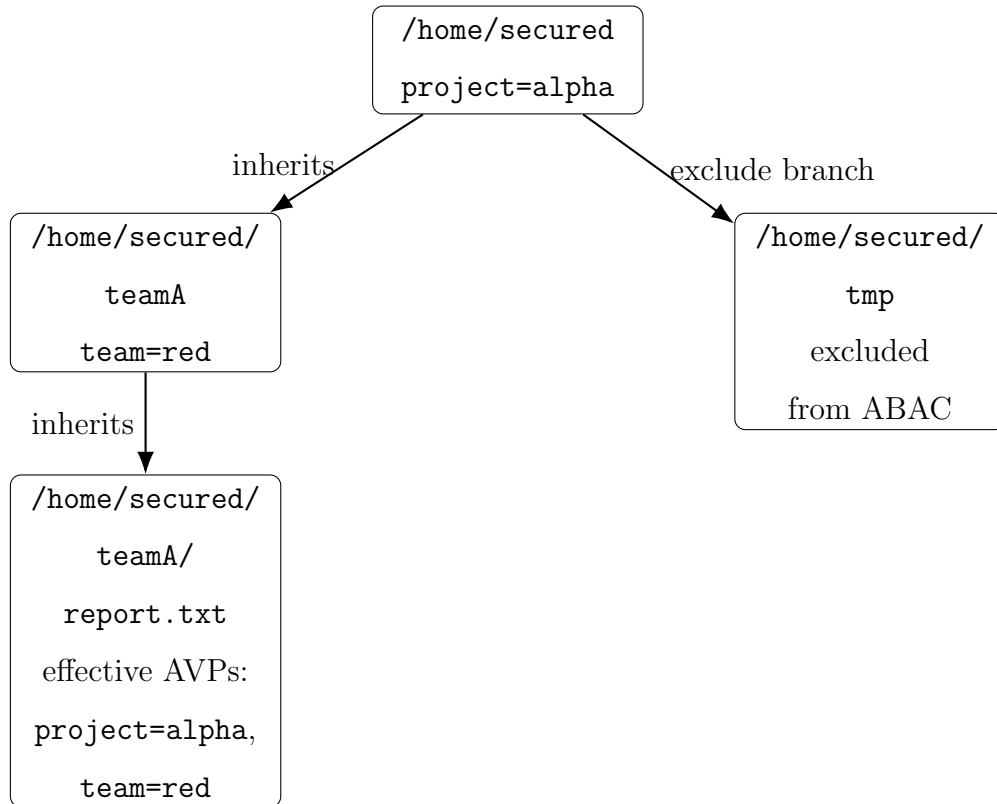


FIGURE 3.5: Directory-level object modeling in the maintained design. Parent directories contribute inherited attributes, while excluded branches leave the ABAC path entirely.

This second design change is equally important. Instead of forcing object attributes to be repeated for every file, the maintained model lets directory labels provide shared context to descendants and lets excluded subtrees opt out cleanly. That reduces configuration size and better matches how administrators reason about real file hierarchies.

3.5 Benchmark Methodology

The benchmarking workflow was designed specifically around the active maintained code path. Synthetic datasets are generated in the textual form accepted by the current kernel parser. The benchmark rewrites generated users so that they correspond to the actual non-root benchmark account on the system, can inject a controlled readable subset of objects, and runs the same workload across the full optimization grid using the same kernel and filesystem state.

This methodology is intentionally strict. The benchmark is meant to answer a narrow question: given the same ABAC system state, how much difference does each optimization make individually, and what happens when the policy-tree and hash-map features are enabled together?

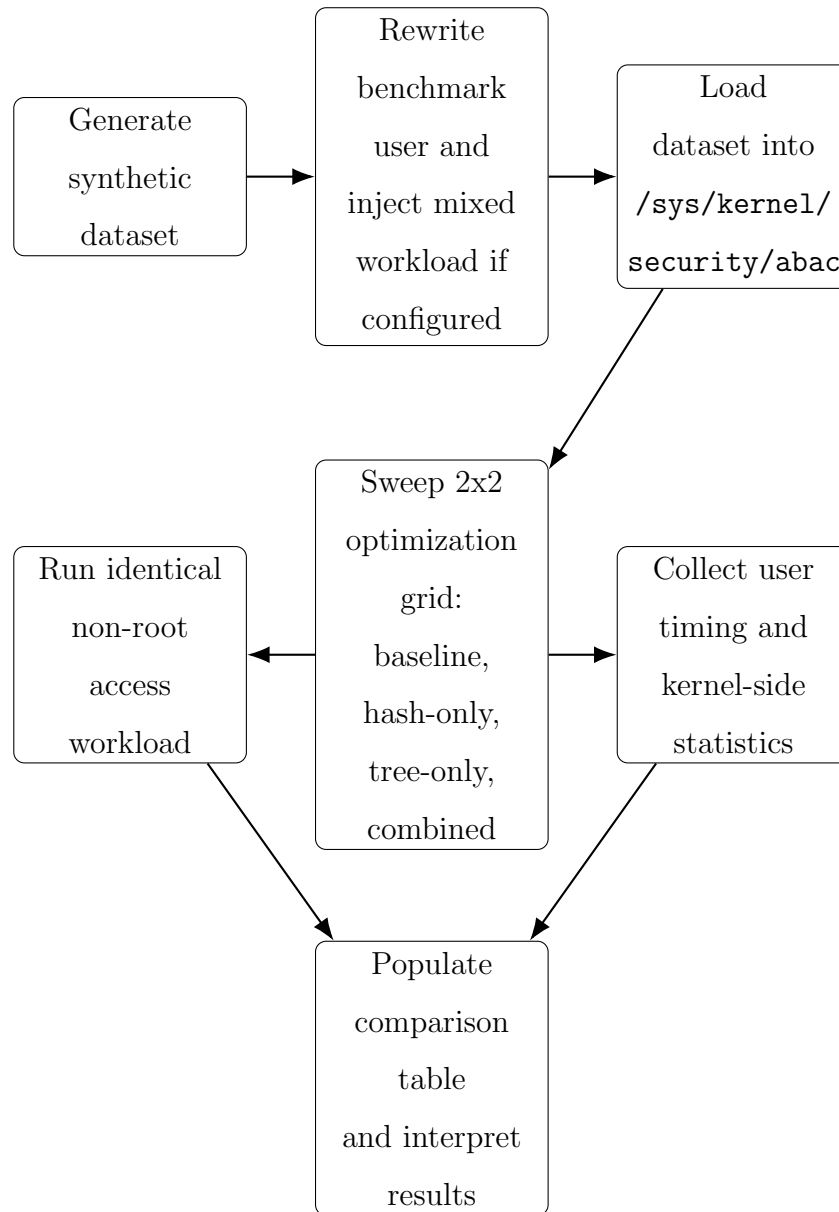


FIGURE 3.6: Benchmark orchestration showing the experimental pipeline around the 2x2 optimization sweep.

	Hash Map OFF	Hash Map ON
Policy Tree OFF	Baseline Neither optimization enabled	Hash-only Hash lookup enabled, tree disabled
Policy Tree ON	Tree-only Compiled tree enabled, hash disabled	Combined Tree and hash both enabled

FIGURE 3.7: Four-configuration optimization grid used for planned performance evaluation.

Chapter 4

Implementation Details

4.1 Kernel Patch and Porting Changes

The kernel patch is not just a transport mechanism; it is the technical record of the port itself. It resolves several concrete incompatibilities between the older ABAC code and Linux ‘6.19.8’.

```
source "security/abac/Kconfig"
obj-$(CONFIG_SECURITY_ABAC) += abac/
```

LISTING 4.1: Kernel build-system integration introduced by the maintained patch

```
ccflags-y := -I$(srctree)/security/abac/include/
obj-$(CONFIG_SECURITY_ABAC) := abac.o

abac-y := avp.o abacfs.o abac.o env.o obj.o pathcfg.o policy.o resolve.o tree.o
```

LISTING 4.2: Maintained ABAC object set built inside the modern kernel tree

This wiring solved the first incompatibility: without these lines, the modern kernel simply did not know ABAC existed, so it could neither expose it in configuration nor compile it.

4.1.1 LSM Registration Port

Another incompatibility appeared in the LSM registration path. The older ABAC code assumed an older registration API in which a string name was passed directly through `security_add_hooks` and the `DEFINE_LSM` block carried older field expectations. On Linux ‘6.19.8’, this no longer matched the expected registration shape.

```
static struct lsm_id abac_lsmid __ro_after_init = {
    .name = "abac",
};

static int __init abac_init(void)
{
    security_add_hooks(abac_hooks, ARRAY_SIZE(abac_hooks), &abac_lsmid);
    return 0;
}

DEFINE_LSM(abac) = {
    .id = &abac_lsmid,
    .init = abac_init,
};
```

LISTING 4.3: Modernized LSM identity and registration path required by the Linux ‘6.19.8’ port

This solved the registration incompatibility by aligning ABAC with the modern in-tree LSM pattern already used by contemporary kernels.

4.1.2 Securityfs Initialization Port

The ABAC `securityfs` initialization path also had to be modernized. The older code used an `initcall` shape that newer kernels reject via stricter type checking. This surfaced as a build-time static assertion failure. The fix was to convert the

filesystem creation routine into a proper integer-returning init function with the correct signature and explicit success/failure return values.

This change matters because `securityfs` is the live control plane of the ABAC system. If initialization is incompatible, the system may compile but fail to expose the interfaces needed for policy, attributes, scope, and instrumentation.

4.1.3 Correctness Fixes Included in the Patch

The patch also includes smaller but important correctness repairs. A good example is the object-path parsing fix in `obj.c`, where the path buffer terminator had to be placed at the correct index. This was important because object attributes are keyed by path; a malformed path copy directly undermines correct policy matching.

4.1.4 Runtime Interface Added Through the Patch

The patch also enlarges the runtime interface beyond the original attribute-loading flow. In addition to `policy`, `user_attr`, `obj_attr`, and `env_attr`, the maintained kernel path now exposes:

- `mode`
- `action`
- `perf`
- `stats`
- `include_paths`
- `exclude_paths`

These additions are what make same-kernel benchmarking, runtime scope management, and instrumentation possible in the maintained implementation.

4.2 Userspace Utilities and Services

The userspace layer is implemented in `abac_utils/`. It provides the `abac` CLI entry point, persistent configuration under `/etc/abac/`, a loader that serializes policy and attributes into `securityfs`, a local object-attribute server, a watcher service, and an environment-update service.

The design follows the service-oriented model described in the original ABAC literature, but extends it operationally. The installer script not only installs the CLI but also initializes the ABAC configuration directory, creates the shared directory and group used by the system, and installs three systemd units: the core object-attribute service, the watcher service, and the environment-update service.

4.3 Negative Rule Support

One of the most important maintained extensions is support for explicit negative rules. The original allow-only model was sufficient for a basic proof of concept, but it was too limited for richer policy expression. The present implementation introduces a distinct effect field in each rule, separating the requested operation from the rule's decision. This allows rules to represent both `ALLOW` and `DENY` behavior while preserving the basic text policy format.

Backward compatibility was preserved deliberately. If a policy line omits the effect field, the parser treats it as an allow rule. This means older policy files remain valid and migration cost remains low.

```
static enum abac_effect parse_rule_effect(const char *token)
{
    if (token && strncmp("DENY", token, 4) == 0)
        return ABAC_DENY;
    return ABAC_ALLOW;
}
```

```
r->op = ABAC_IGNORE;
if (effect_delim_index == -1) {
    parse_rule_op(buffer + env_delim_index + 1, &r->op);
    r->effect = ABAC_ALLOW;
} else {
    parse_rule_op(buffer + env_delim_index + 1, &r->op);
    r->effect = parse_rule_effect(buffer + effect_delim_index + 1);
}
```

LISTING 4.4: Parser extension for explicit rule effects while preserving backward compatibility

The complexity here is not just syntactic. This change introduced an explicit conflict-resolution dimension into the policy model, enabling exception-style policies without breaking older rule files already accepted by the repository.

4.4 Compiled Policy Tree and Runtime Optimization Flags

The current maintained implementation exposes tree-based evaluation and hash-map indexed lookup as separate optimization controls. Conceptually, this creates four resolver configurations:

- neither optimization enabled
- only the policy tree enabled
- only the hash-map lookup enabled
- both optimizations enabled together

The policy tree is a compiled grouping structure organized by object bucket, operation bucket, user/environment matcher, and effect leaf. This makes the maintained code path easier to reason about while still achieving the main goal of tree-based evaluation: reducing repeated work relative to full rule-list scanning. In the maintained implementation, this tree can be paired with hash-map indexed user and object lookup so that the resolver does not spend unnecessary time linearly traversing those collections before tree evaluation begins.

```

while (rule != NULL) {
    obj_bucket = find_object_bucket(tree, rule->object);
    if (!obj_bucket) {
        obj_bucket = kcalloc(1, sizeof(*obj_bucket), GFP_KERNEL);
        obj_bucket->object = clone_avp_list(rule->object);
        obj_bucket->next = tree->objects;
        tree->objects = obj_bucket;
    }

    op_bucket = find_op_bucket(obj_bucket, rule->op);
    if (!op_bucket) {
        op_bucket = kcalloc(1, sizeof(*op_bucket), GFP_KERNEL);
        op_bucket->op = rule->op;
        op_bucket->next = obj_bucket->ops;
        obj_bucket->ops = op_bucket;
    }

    match = find_match_node(op_bucket, rule->user, rule->env);
    if (!match) {
        match = kcalloc(1, sizeof(*match), GFP_KERNEL);
        match->user = clone_avp_list(rule->user);
        match->env = clone_avp_list(rule->env);
        match->next = op_bucket->matches;
        op_bucket->matches = match;
    }

    append_effect(match, rule->effect);
    rule = rule->next;
}

```

```
}
```

LISTING 4.5: Compiled policy-tree construction in the maintained resolver

This is one of the most technically meaningful additions in the repository. Instead of evaluating the same object and operation predicates over and over again across a flat linked list, the code compiles policy into a grouped structure that prunes repeated work. Together with hash-map indexed retrieval of user and object state, this is the key reason tree mode can outperform the original sequential scan.

Algorithm 1 Access Decision Workflow in the Maintained ABAC LSM

```

1: procedure ABAC_FILE_PERMISSION(file, mask)
2:   uid  $\leftarrow$  current user id
3:   if uid < 1000 then
4:     return allow
5:   end if
6:   path  $\leftarrow$  path of accessed file
7:   if path is outside configured ABAC scope then
8:     return allow
9:   end if
10:  if policy or required attributes are not initialized then
11:    return deny
12:  end if
13:  if recording is enabled then
14:    start timer
15:  end if
16:  decision  $\leftarrow$  resolve(uid, path, mask) using current resolver mode
17:  if recording is enabled then
18:    update kernel timing statistics
19:  end if
20:  if decision is allow then
21:    return allow
22:  else
23:    return deny
24:  end if
25: end procedure

```

```

static int abac_file_permission(struct file *file, int mask)
{
    u64 start_ns, elapsed_ns;
    bool allowed;

```

```

    unsigned int UID = current_uid().val;
    if (UID < 1000) {
        return 0;
    }
    ...
    if (!abac_path_is_covered(path)) {
        kfree(buff);
        return 0;
    }
    if (policy == NULL || user_attr == NULL || obj_attr == NULL) {
        kfree(buff);
        return -EPERM;
    }
    start_ns = abac_recording ? ktime_get_ns() : 0;
    allowed = abac_resolve(UID, path, mask);
    if (abac_recording) {
        elapsed_ns = ktime_get_ns() - start_ns;
        abac_record_decision(elapsed_ns, allowed);
    }
    ...
}

```

LISTING 4.6: Actual LSM enforcement hook in the maintained kernel path

This hook is where most of the maintained feature set meets: path scope, resolver dispatch, policy initialization checks, and performance recording.

```

static ssize_t mode_write(struct file *filp, const char __user *buffer,
                          size_t len, loff_t *off)
{
    char mode[16];
    size_t copy_len = min(len, sizeof(mode) - 1);
    ...
    if (strcmp(mode, "linear") == 0) {
        abac_mode = ABAC_RESOLVE_LINEAR;
        return len;
    }
}

```

```

    }
    if (strcmp(mode, "tree") == 0) {
        abac_mode = ABAC_RESOLVE_TREE;
        return len;
    }
    return -EINVAL;
}

```

LISTING 4.7: Runtime resolver-mode switching through `securityfs`

This switch avoids maintaining multiple kernels while still preserving fair resolver comparison. The hash-map feature is treated in the same spirit as an independent optimization flag, so the evaluation framework can benchmark neither optimization, each optimization in isolation, and both together.

4.5 Path Scope Support

The original implementation assumed a fixed protected directory. This was a serious practical limitation: it meant that changing protected scope effectively required changing kernel-side assumptions. The maintained implementation resolves this by introducing include/exclude prefix lists exposed through `securityfs` and persisted from userspace.

```

def load_path_scope():
    config_path = CONFIG_ROOT + CONFIG_PATH_SCOPE_FILE
    include_kernel_path = ABAC_MOUNT + KERN_INCLUDE_PATHS_FILE
    exclude_kernel_path = ABAC_MOUNT + KERN_EXCLUDE_PATHS_FILE
    ...
    for path in scope.get("include", []):
        include_content += f"{str(Path(path).resolve())}\n"
    for path in scope.get("exclude", []):
        exclude_content += f"{str(Path(path).resolve())}\n"

```

```
with open(include_kernel_path, 'w') as f:
    f.write(include_content)
with open(exclude_kernel_path, 'w') as f:
    f.write(exclude_content)
```

LISTING 4.8: Userspace scope loading that expands the old single-directory model into configurable global coverage

This userspace step is what makes the globalized coverage model operational. Instead of compiling a single directory assumption into the kernel, administrators can now project arbitrary directory sets into the kernel at runtime through `include_paths` and `exclude_paths`.

```
bool abac_path_is_covered(const char *path)
{
    bool included = true;

    if (!path || path[0] != '/')
        return false;
    if (abac_include_paths)
        included = path_matches_entries(path, abac_include_paths);
    if (!included)
        return false;
    if (abac_exclude_paths && path_matches_entries(path, abac_exclude_paths))
        return false;
    return true;
}
```

LISTING 4.9: Kernel-side coverage check used after the move from one fixed directory to include/exclude-based runtime scope

This kernel-side check is what makes the userspace scope-loading step effective. The include/exclude lists loaded from userspace become part of the actual enforcement path here.

4.6 Directory-Level Object Attribute Inheritance

The second major usability extension is resolver-time directory inheritance. Instead of requiring every file to be annotated independently, the resolver can now build the effective object AVP set by walking path prefixes and merging attributes from parent directories down to the requested object. More specific paths override less specific ones for the same attribute name, while unrelated attributes accumulate. This preserves the original path-based object model while making it far more usable for realistic directory trees.

```
for (i = 1; i < path_len; i++) {
    if (path[i] != '/')
        continue;
    strncpy(prefix, path, i);
    prefix[i] = '\0';
    attrs = get_obj_avp(prefix, head);
    if (!attrs)
        continue;
    if (!merge_avp_list(&merged, attrs)) {
        destroy_avp_list(merged);
        return NULL;
    }
}

attrs = get_obj_avp(path, head);
if (attrs && !merge_avp_list(&merged, attrs)) {
    destroy_avp_list(merged);
    return NULL;
}
```

LISTING 4.10: Resolver-time path-prefix walk for directory-level attribute inheritance

This feature has high practical impact despite looking simple. It changes the administrative model from per-file labeling to hierarchical labeling, which greatly

reduces policy-management effort in realistic directory trees while keeping the source of truth compact.

4.7 Kernel-Side Instrumentation

The current repository adds explicit runtime controls and statistics files in `/sys/kernel/security/abac` for benchmarking and inspection. The `mode` file switches between `linear` and `tree`. The `action` file controls recording. The `perf` file exposes the last decision time. The `stats` file exposes aggregate counters including total time, minimum and maximum time, decision count, allow count, deny count, and previous access time.

This instrumentation is intentionally lightweight. It does not attempt to store a full history of decisions. Instead, it exposes exactly the information needed for comparative benchmarking with minimal measurement overhead.

4.8 Benchmark Harness Implementation

The benchmark harness in `perf_eval/` is the concrete implementation of the evaluation methodology described earlier. It generates synthetic datasets in the maintained parser format, rewrites dataset users to the actual benchmark user, can inject controlled readable subsets for mixed workloads, and orchestrates same-kernel comparisons over the live `securityfs` interface.

```
ensure_secured_files(config["obj"]["count"], args.run_as_user)
rewrite_dataset_user_for_runner(dataset_dir, args.run_as_user, benchmark_config)
inject_mixed_workload(dataset_dir, benchmark_config)
load_dataset(dataset_dir)
```

LISTING 4.11: Benchmark preparation that rewrites the dataset for the real benchmark user

```
(ABAC_ROOT / "mode").write_text(mode + "\n")
(ABAC_ROOT / "action").write_text("RESET\n")
(ABAC_ROOT / "action").write_text("RECORD\n")
user_stats = run_access_test(
    run_as_user,
    config["obj"]["count"],
    config["iterations"],
    config.get("seed", 42),
)
(ABAC_ROOT / "action").write_text("STOP\n")
kernel_stats = parse_stats()
```

LISTING 4.12: Per-mode benchmark orchestration using the live kernel interface

The main feature here is methodological control: the same kernel, same data, same files, and same benchmark user are reused across both modes, so resolver structure remains the variable under test.

4.9 Files Central to the New Features

The maintained kernel-side work is centered on `abac.c`, `abacfs.c`, `policy.c`, `resolve.c`, `tree.c`, and `pathcfg.c`, together with their associated headers. These files implement the LSM hook, the `securityfs` runtime interface, policy parsing, flag-controlled rule resolution, policy-tree compilation, hash-indexed state lookup, and path-scope parsing.

The userspace side is centered on `abac_utils/src/load.py`, `scope.py`, `server.py`, `watch.py`, `env_update.py`, and `abac_init.py`. These components persist configuration, load live kernel state, validate object updates against configured scope, monitor watched include directories, and refresh environment attributes.

TABLE 4.1: Feature-wise contribution map for the maintained ABAC Fedora port

Feature	Main files	Primary impact
Modern kernel port	<code>kernel_patch</code> , <code>abac.c</code> , <code>abacfs.c</code>	Makes ABAC buildable and bootable on Linux ‘6.19.8’
Negative rules	<code>policy.c</code> , <code>policy.h</code> , <code>load.py</code>	Adds explicit deny semantics and exception-style policy expression
Compiled policy tree	<code>tree.c</code> , <code>resolve.c</code>	Reduces repeated rule evaluation and improves resolver efficiency
Hash-map indexes	<code>user.c</code> , <code>obj.c</code> , <code>resolve.c</code>	Eliminates linear lookup over user and object state before policy evaluation
Runtime mode switching	<code>abacfs.c</code> , <code>abacfs.h</code>	Enables fair same-kernel comparison of linear and tree
Path scope	<code>pathcfg.c</code> , <code>scope.py</code> , <code>load.py</code>	Replaces hard-coded coverage with configurable include/exclude control
Directory inheritance	<code>resolve.c</code>	Makes hierarchical policy administration practical for real directory trees
Kernel instrumentation	<code>abac.c</code> , <code>abacfs.c</code>	Exposes decision-time statistics for resolver evaluation
Benchmark harness	<code>generate_dataset.py</code> , <code>benchmark.py</code> , <code>access_test.py</code>	Produces reproducible mixed allow/deny workloads on the active code path

4.10 Design Decisions

Six design decisions dominate the maintained implementation.

First, the current policy language was preserved rather than replaced. This reduced migration cost and allowed the new features to be added around the existing administrative model. Second, resolver comparison was made runtime-switchable rather than split into multiple maintained kernels. This improves fairness and

maintainability. Third, path scope remained path-based, because prefix lists fit the rest of the design well. Fourth, directory inheritance was implemented inside the resolver rather than flattened into userspace state, avoiding duplication and stale expansion. Fifth, hash-map indexing was designed to be controlled independently of the policy tree so that its isolated contribution can be benchmarked instead of being folded invisibly into tree-mode results. Sixth, kernel timing was kept separate from user timing so that the benchmark could distinguish authorization cost from larger system overheads.

4.11 Challenges Encountered

Several implementation challenges shaped the final design, and many of them were operational rather than purely algorithmic.

The first challenge was kernel-port maintainability itself. Working directly inside a full modified kernel tree makes recovery difficult: a failed build, a partial cleanup, or an experimental edit can leave the developer with a large amount of unstructured state that is hard to reason about. Preserving the maintained kernel work as a patch solved an important practical problem here. It meant that progress could always be reconstructed against a clean Linux ‘6.19.8’ tree, kernel-side deltas remained auditable, and porting work did not depend on preserving a single long-lived local source tree in perfect condition.

The second challenge was the behavior of kernel builds under heavy parallelism. Full kernel compilation can consume significant CPU and memory, and aggressive use of parallel `make` jobs may appear to hang or stall when processor cores are saturated, when link-heavy phases serialize, or when system responsiveness drops under load. In practice, this affected the development workflow because port validation was not just about code correctness; it also depended on choosing build settings that remained stable on the available machine. This made incremental rebuilds, careful

patch-based recovery, and a bounded kernel diff more valuable than they would be in a smaller userspace-only project.

The third challenge was API drift across kernel versions. Modern kernel expectations around LSM registration and `securityfs` initialization are stricter than those assumed by the older ABAC code path. These were not stylistic mismatches; they produced concrete build failures and required the registration path, initcall signatures, and related integration logic to be rewritten into the forms expected by Linux ‘6.19.8’.

The fourth challenge was balancing experimental fairness with implementation simplicity. Resolver comparison is only meaningful if the same kernel, same policy language, same attribute state, same files, and same benchmark user are held fixed across runs. That requirement shaped the design of both the runtime mode switch and the planned hash-map flag behavior, because it ruled out looser workflows in which each optimization variant would evolve or be measured separately.

The fifth challenge was reconciling global scope configuration with realistic administration. Moving from one fixed protected directory to include/exclude-controlled runtime scope makes the system much more flexible, but it also introduces new consistency concerns. Userspace services that watch for object updates can only monitor configured paths meaningfully, so the report’s final design had to keep kernel scope, persisted configuration, and service behavior aligned.

Finally, directory inheritance had to be added without exploding configuration size or pushing unnecessary work into userspace. Flattening inherited attributes ahead of time would have duplicated state and increased the risk of stale expansions. Implementing inheritance inside the resolver kept the persisted configuration compact, but required more care in path-prefix walking, merge precedence, and correctness of object-path handling.

4.12 Hash-Map Indexed User and Object Lookup

The compiled policy tree removes the most obvious full-policy linear scan from the original implementation, but tree-based evaluation alone is not sufficient if surrounding state access remains list-based. For that reason, the maintained implementation introduces hash-map-backed indexes beside the policy tree for user and object lookup. This hash layer is intended to be benchmarked with the same style of runtime flag control used for the tree feature, so that its isolated contribution can be measured cleanly.

This does not replace the tree. Instead, the two optimizations are complementary. The tree improves the structure of policy evaluation itself, while hash maps improve access to the user and object data needed before tree evaluation begins. In practical terms, this means the resolver now reduces traversal overhead at two distinct stages: state retrieval and rule evaluation.

```
static avp *get_user_avp(unsigned int UID, struct hlist_head *user_map)
{
    abac_user *cursor;
    hash_for_each_possible(user_map, cursor, node, UID) {
        if (cursor->uid == UID)
            return cursor->attrs;
    }
    return NULL;
}

static avp *get_obj_avp(char *path, struct hlist_head *obj_map)
{
    abac_obj *cursor;
    u32 key = jhash(path, strlen(path), 0);

    hash_for_each_possible(obj_map, cursor, node, key) {
        if (strcmp(path, cursor->path) == 0)
            return cursor->attrs;
    }
}
```

```
    }  
    return NULL;  
}
```

LISTING 4.13: Hash-map indexed user and object lookup used beside the policy
tree

This is an important maintained feature because it ensures that the resolver's gains are not limited to the policy data structure alone. The tree reduces repeated policy traversal, while the hash-map layer removes the need to scan user and object collections linearly before that tree is even consulted.

Chapter 5

Results

5.1 Functional Validation

Functional testing confirmed that the ported kernel boots correctly and that ABAC appears in the active Linux Security Module list. The expected `/sys/kernel/security/abac` directory is created, and the policy, user attribute, object attribute, and environmental attribute interfaces are exposed as expected. Userspace services were successfully installed and started, showing that the control plane works together with the kernel-side interface.

5.2 Authorization Behavior

End-to-end policy validation was performed using a non-root user and protected files under `/home/secured`. A matching `READ` rule allowed file reads, while write attempts were denied when no matching `MODIFY` rule existed. Additional tests confirmed that excluded paths bypass ABAC enforcement as intended, and that directory-level object attributes are inherited by child files and subdirectories, demonstrating that the resolver-time inheritance logic behaves correctly.

5.3 Performance Evaluation

Performance evaluation is organized around the four-way optimization grid supported by the maintained design. The benchmark uses the same data, the same kernel, the same file set, the same benchmark user, and the same access pattern, while toggling the policy-tree and hash-map features independently.

TABLE 5.1: Optimization-grid performance results from the maintained Fedora port

Configuration	Mean user-observed access latency	Kernel-side decision time
Neither tree nor hash map	22,896.65 ns	15,970,257 ns
Tree only	16,962.23 ns	13,483,783 ns
Hash map only		
Tree and hash map		

From the currently available measurements, enabling the policy tree while leaving the hash-map layer disabled reduced mean user-observed access latency by approximately 25.9%. Kernel-side decision time decreased by approximately 15.6%. The remaining grid entries are left blank here so that future benchmarking can record the isolated effect of hash-map indexing and the combined effect of both optimizations under the same methodology.

TABLE 5.2: Interpretation of the maintained feature set by technical impact

Feature	Why it is technically important
Negative rules	Introduces an explicit conflict dimension into policy semantics and enables deny-overrides behavior
Compiled policy tree	Converts repeated per-request list scanning into grouped in-memory evaluation over object and operation buckets
Hash-map indexes	Remove surrounding linear traversal for user and object lookup before policy evaluation begins
Path scope	Moves coverage from a compile-time constant to runtime-managed kernel state
Directory inheritance	Changes the administrative model from per-file labeling to hierarchical AVP composition
Kernel instrumentation	Separates authorization cost from broader userspace and VFS timing noise
Benchmark harness	Provides a same-kernel methodology that attributes measured differences to resolver structure rather than environment drift

5.4 Discussion of Results

The performance numbers should not be treated as directly numerically comparable to the earlier papers because the kernel version, resolver implementation details, benchmark harness, and workload construction differ. However, the broader conclusion remains aligned with the prior literature. The ABAC architecture remains viable on a modern kernel, and the maintained resolver gains come from a

combination of structured tree-based evaluation and hash-map indexed retrieval of the user and object state needed by that evaluation.

The functional results are equally important. The repository is not only a performance experiment. It is also a complete modernized ABAC stack that integrates kernel patching, runtime control, userspace loading, scope management, inheritance, and service wiring into one maintainable system.

Chapter 6

Conclusion

6.1 Key Outcomes

This report has shown that the ABAC Fedora port preserves the essential architecture of prior ABAC-LSM research while extending it into a more practical and modern system. The kernel-side work was successfully ported into a Linux ‘6.19.8’ patch-based integration flow. The userspace layer was expanded to support configuration management, scope loading, object-attribute updates, watching, and environmental refresh logic. The maintained code path now supports negative rules, a compiled policy-tree resolver, hash-map indexed user and object lookup, runtime optimization-flag comparison, configurable scope, directory inheritance, and kernel-side instrumentation.

The evaluation results confirm both correctness and benefit. The system enforces policy as intended, supports the richer operational behavior introduced by the maintained extensions, and demonstrates that tree-based evaluation reduces authorization overhead relative to linear scanning.

6.2 Next Steps

The clearest next steps now lie beyond the resolver baseline itself, for example in broadening workload diversity, evaluating larger policy corpora, and refining the balance between instrumentation detail and measurement overhead. Since the maintained implementation already combines compiled tree-based policy evaluation with hash-map indexed user and object lookup, future work can focus on quantifying how those choices behave under larger and more heterogeneous deployments.

More broadly, the repository is now in a position where it can support both operational use and future research. It retains continuity with the original ABAC-LSM literature while offering a clearer maintained implementation surface for additional optimization and evaluation.

Bibliography

- [1] H.O. Sai Varshith, Shamik Sural, Jaideep Vaidya, and Vijayalakshmi Atluri. Enabling attribute-based access control in linux kernel. In *Proceedings of the 2022 ACM on Asia Conference on Computer and Communications Security*, ASIA CCS '22, pages 1237–1239, New York, NY, USA, 2022. Association for Computing Machinery.
- [2] H. O.Sai Varshith, Shamik Sural, Jaideep Vaidya, and Vijayalakshmi Atluri. Efficiently supporting attribute-based access control in linux. *IEEE Transactions on Dependable and Secure Computing*, 21(4):2012–2026, 2024. Publisher Copyright: © 2023 IEEE.